

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
```

```

<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>

```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```

<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>

```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

*I, **Aaron Samuel S(192411022)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some	Basic analysis with weak reasoning	No meaningful analysis provided

			justification		
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1. HTTP Response Analysis

Given Header

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

1. Identify any incorrect or invalid fields in the header.

- **Content-Length: -150** is invalid because Content-Length cannot be negative.
- The **Date** format appears valid, although the actual day should match the calendar date.

2. Analyze the issue with Content-Length and explain its impact.

- **Content-Length** specifies the size of the response body in **bytes**.
- A negative value is not allowed by HTTP standards.
- Effects:
 - Browser cannot determine the correct response size.
 - Response may be rejected.
 - Data transfer may fail or be truncated.
 - Connection handling becomes unreliable.

3. Determine whether the status code matches the response correctness.

- **Status Code: 200 OK** indicates that the request was successful.
- Since the response contains an invalid header (negative Content-Length), the response is technically malformed.
- Therefore, the status code does **not** fully match the correctness of the response.

4. Suggest corrections for the identified errors.

Correct header:

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html
Content-Length: 150
Connection: keep-alive

Corrections:

- Replace **-150** with the actual content size.
 - Ensure all header values follow HTTP specifications.
-

2. DOM Structure Analysis

Given DOM

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

1. Identify missing or improperly closed elements.

Missing closing tag:

- `</p>`

The paragraph is opened but never closed.

2. Analyze how the browser will interpret this structure.

Modern browsers automatically correct HTML errors.

The browser will interpret it as:

```
<div>
<p>HelloWorld</p>
</div>
```

The browser automatically inserts the missing `</p>` before closing the `<div>`.

3. Determine the impact on rendering and layout.

- Page usually displays correctly.
- Browser performs automatic DOM correction.
- Complex pages may have unexpected layouts.
- JavaScript DOM manipulation may behave differently.

4. Suggest corrections to make the DOM valid.

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld</p>
</div>
</body>
</html>
```

3. GET vs POST in Login System

1. Interpret the differences in data transmission between GET and POST.

GET	POST
Sends data through URL	Sends data in request body
Data is visible	Data is hidden from URL
Limited data size	Supports larger data
Can be bookmarked	Cannot be bookmarked
Used for retrieving data	Used for submitting data

2. Analyze which method is more suitable for login and why.

POST is more suitable because:

- Username and password are not exposed in the URL.
- Credentials are not stored in browser history.
- More secure when combined with HTTPS.
- Designed for submitting sensitive information.

3. Identify security risks in using GET for authentication.

Risks include:

- Password appears in URL.
- Browser history stores credentials.
- Server logs may record credentials.
- URLs can be shared accidentally.
- Credentials may leak through referrer headers.

4. Suggest best practices for secure login implementation.

- Use **POST** instead of GET.
 - Always use **HTTPS**.
 - Hash passwords before storing them.
 - Use secure session tokens.
 - Implement strong password policies.
 - Enable Multi-Factor Authentication (MFA).
 - Validate and sanitize user inputs.
-

4. HTML Code Inspection

Given HTML

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
```



```
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

1. Identify missing closing tags.

Missing closing tags:

- `</h1>`
- `</p>`

2. Analyze issues with the `<img>` tag.

- `<img>` is a **void element**, so it does not require a closing tag.
- However, it should include an **alt** attribute for accessibility.

Example:

```

```

3. Determine how browsers will handle these errors.

Browsers automatically:

- Close the `<h1>` tag.
- Close the `<p>` tag.
- Render the image normally.

The page usually displays correctly, but invalid HTML may cause layout inconsistencies.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
```

```
<h1>Welcome</h1>

<p>This is a paragraph</p>



</body>
</html>
```

5. Browser-Server Communication Flow

1. Interpret each step in the communication flow.

Step	Explanation
1. User enters URL	User types a website address into the browser.
2. DNS Lookup	DNS converts the domain name into an IP address.
3. HTTP Request	Browser sends a request to the web server.
4. Server Processing	Server processes the request and retrieves required resources.
5. HTTP Response	Server sends HTML, CSS, JavaScript, images, and other resources back to the browser.
6. Browser Rendering	Browser builds the DOM, applies CSS, executes JavaScript, and displays the webpage.

2. Identify where delays can occur.

Delays may occur during:

- DNS lookup
- Network latency
- Server processing
- Database queries
- Downloading large files
- Browser rendering and JavaScript execution

3. Analyze the role of DNS in the process.

DNS (Domain Name System):

- Translates the website name into an IP address.
- Enables the browser to locate the correct server.
- Without DNS, users would need to enter numerical IP addresses instead of domain names.

4. Suggest optimizations to improve performance.

Performance improvements include:

- Use DNS caching.
- Enable browser caching.
- Compress files using Gzip or Brotli.
- Use a Content Delivery Network (CDN).
- Minify HTML, CSS, and JavaScript.
- Optimize images.
- Reduce HTTP requests.
- Use HTTP/2 or HTTP/3.
- Optimize server and database performance.